MoCafe v1.20 — A Monte-Carlo Dust RT Code: Scattering and Polarization User Guide

Abstract

MoCafe is a parallel (Fortran 90 + MPI) Monte-Carlo radiative-transfer code for dust scattering. It follows monochromatic photon packets emitted by a source through a dusty medium, scatters them off dust grains (Henyey–Greenstein phase function or a tabulated Mueller matrix, with full Stokes polarization), and "peels off" a contribution toward each observer at every scattering site to build a FITS or HDF5 image. The medium is a Cartesian grid, a clumpy population of discrete dust spheres, or an adaptive octree fed by RAMSES or Illustris-TNG. Two single-run scans turn one Monte-Carlo history into an image cube over a grid of albedo, scattering asymmetry, and optical depth. This guide covers installation, execution, the complete input reference, the scans, the clumpy and AMR media, the converters, the output products, worked examples, and the underlying algorithms.

Contents

1. Overview

MoCafe follows photon packets emitted by a source through a dusty medium. Packets scatter off dust grains and, at every scattering site, a contribution is "peeled off" toward each observer to build an image of the source seen through the dust. Transport is *monochromatic*: the dust is grey, with a single albedo, asymmetry parameter, and extinction cross section.

Capabilities.

- **Three media**, selected by `par%grid_type`: a uniform or structured **Cartesian** grid (`'car'`), a **clumpy** medium of discrete non-overlapping dust spheres (`'clump'`), and an **adaptive octree** read from a generic AMR file (`'amr'`).
- **Internal and external sources**: point, uniform, uniform in (x, y), Gaussian and exponential distributions, and isotropic external illumination through a spherical, cylindrical, or rectangular boundary.
- **Stokes polarization** through a tabulated Mueller-matrix phase function.
- **Single-run scans** (one Monte-Carlo run → many images): the (a, g) scan of Seon (2010) and the polychromatic optical-depth scan of Jonsson (2006), which combine into `scatt`(x, y, a, g, τ).
- **Quasi-random photon launching** (`par%launch_sequence`) for lower launch noise in extended-source and externally illuminated runs.
- **HDF5 and FITS** output through a format-agnostic facade (`par%file_format`), plus Python readers and converters.
- **MPI** parallelism in a master–slave or an equal-share mode, with MPI-3 shared memory for the read-only grid, clump, and octree arrays (one copy per node).

Units. MoCafe is unit-agnostic: the output image is in `(luminosity unit)` $\mathrm{cm}^{-2}\,\mathrm{sr}^{-1}$, normalized to `par%luminosity`. A physical length scale enters only through `par%distance_unit` when the dust column is set from a density in cm^{-3}; a dimensionless model instead fixes the column with `par%taumax` or `par%tauhomo`. The system center is always at the origin.

2. Installation

2.1 Prerequisites

- A Fortran 2008 MPI compiler: Intel oneAPI (`mpiifort` or `mpiifx`) or GNU (`mpif90`). MoCafe is MPI-only.
- **CFITSIO** (required).
- **HDF5** 1.14+ with Fortran bindings built by the same compiler (optional; on by default).
- Python 3 with `numpy`, `astropy`, `h5py` (and `scipy` for the Illustris converter) for the analysis tools and the converters.

2.2 Building MoCafe

```
make                              # HDF5=1 (default) -> MoCafe.x
make HDF5=0                       # CFITSIO only
make HDF5_PREFIX=/usr/local/hdf5  # override HDF5 location
make F90=mpif90                   # force a compiler
make DEBUG=1                      # checked / traceback flags
```

The compiler is auto-selected `mpiifort` → `mpiifx` → `mpif90`. The build always passes `-cpp -DMPI` and (by default) `-DHDF5`. Objects and module files are written into `src/`; only `MoCafe.x` is placed at the top level. When you add a source file, append its object to `OBJS` in the `Makefile` in dependency order. Without `-DHDF5` the HDF5 backend compiles to stubs: the link still succeeds, but selecting `par%file_format = 'hdf5'` at run time then fails.

3. Running MoCafe

```
mpirun -np <N> ./MoCafe.x <input>.in
```

The single argument is a Fortran namelist file. Every tunable quantity is a field of the global `par` structure and is set as `par%<name> = <value>` inside a `&parameters / . . . /` block. Sample inputs and `run.sh` scripts live under `examples/`. The default of every parameter is the initializer of `params_type` in `src/define.f90`, which is the authoritative reference.

Reproducibility. A run is bit-identical when `par%iseed` is a fixed non-zero value, `par%use_master_slave` is `.false.` (equal-share), and the number of MPI ranks is fixed (`par%iseed` = 0 seeds from the clock/PID). The default master–slave mode hands out photon batches dynamically, so which rank draws which photon — and therefore the result at the last bit — varies from run to run even at a fixed seed and rank count; use equal-share whenever exact reproducibility matters.

4. Input Parameter Reference

Parameters are grouped below. Types: `L` logical, `I` integer, `R` real, `S` string, `[ ]` array.

4.1 Simulation control

Parameter	Default	Description
`no_photons`	1e5	Number of photon packets
`no_print`	1e7	Progress-print interval (photons)
`iseed`	0	RNG seed (0 → seeded from clock/PID)
`launch_sequence`	'random'	'sobol' = quasi-random (Owen-scrambled Sobol) launch of the emission direction and the external-sphere entry point; transport draws stay pseudo-random (§4.2)

Parameter	Default	Description
qmc_seed	12345	Scramble seed for 'sobol'; a different seed is an independent replicate
luminosity	1.0	Source luminosity (the image is normalized to it)
use_master_slave	.true.	MPI mode: master–slave vs. equal-share
num_send_at_once	10000	Photon batch size (master–slave)
use_reduced_wgt	.true.	Forced first scattering with the reduced-weight scheme

4.2 Quasi-random photon launching

With par%launch_sequence = 'sobol' the *launch* variables of each packet are taken from an Owen-scrambled Sobol point indexed by the global photon number, while every draw after launch (forced first scattering, Henyey–Greenstein deflections, free paths) stays on the Mersenne Twister; the transport estimator is unchanged and unbiased. In v1.20 the quasi-random coordinates are the emission direction (Sobol dimensions 4–5) and the external-sphere entry point (dimensions 6–7); dimensions 1–3 are reserved so that the layout matches v2.00. par%qmc_seed selects the scramble, and several seeds give independent replicates for error estimation.

The gain is configuration-dependent. A point source benefits only indirectly, through a more uniform distribution of forced-first-scattering sites, because its direct image is already noise-free. Extended sources and, especially, external illumination benefit the most: uniform coverage of the emission positions and of the boundary entry points removes the large-scale launch noise from both the direct and the scattered fields. Deep, scattering-dominated clouds gain little, since the multiple-scattering random walk dominates and stays pseudo-random.

Quasi-random launching is rejected at setup (a fatal configuration error) in combination with par%use_stokes, the (a, g) or τ scans, and par%radiation_angular_PDF_file (its angular table is alias-sampled, which breaks the monotone inverse-CDF mapping a stratified coordinate needs).

MPI-count dependence of the external direct image. Because the launch set is indexed by the global photon number, it does not depend on how the photons are distributed over ranks, and the Direct0 image of an internal point source is bit-identical for any -np. The external direct image is different: it rides the peeling_direct_external_sph2 estimator, which draws its own pseudo-random line of sight (by design, not a defect). The external Direct0 therefore still **depends on the number of MPI tasks**, although its total is conserved. Design and validation record: docs/QUASI_RANDOM_LAUNCH_MOCAFE.md.

4.3 Grid geometry (Cartesian)

Parameter	Default	Description
nx, ny, nz	1, 1, 11	Cells per axis
nr	-999	If > 1, shorthand for nx=ny=nz=nr
xmax, ymax, zmax	1.0	Half-extent per axis; the box spans $[-\max, \max]$

Parameter	Default	Description
rmax	-999	Sphere radius; if > 0 and not a cylinder, the box becomes a cube of side $r_{\max}$ and the density vanishes for $r > r_{\max}$
rmin	0.0	Inner cavity radius (shell or clump cavity)
geometry	' '	'', 'sphere', 'cylinder', or 'Plummer' (otherwise a uniform or file-defined box)
xyz_symmetry	.false.	Use one octant of the box (disabled if any $n = 1$, and not used for peel-off)
xy_periodic	.false.	Periodic slab in x, y
z_symmetry	.false.	Mirror symmetry about $z = 0$

par%read_input rewrites some of these: par%rmax > 0 with a non-cylindrical geometry forces a cube with $n_x = n_y = n_z = \max(n_x, n_y, n_z)$ and turns an external source into external_sph, while geometry='cylinder' forces $n_x = n_y$ and external_cyl.

4.4 Grid type

Parameter	Default	Description
grid_type	'car'	'car' (Cartesian), 'clump' (clumpy), 'amr' (octree)
use_clump_medium	.false.	Legacy switch, mapped to grid_type='clump'
use_amr_grid	.false.	Legacy switch, mapped to grid_type='amr'

4.5 Distance unit

Parameter	Default	Description
distance_unit	'kpc'	'kpc', 'pc', 'au', or '' (dimensionless code units)
distance2cm	(from the unit)	cm per code length unit

For a dimensionless model with no density file, par%distance_unit is forced to '' and the dust column is set by par%taumax or par%tauhomo.

4.6 Source geometry

Parameter	Default	Description
source_geometry	'point'	'point', 'uniform', 'uniform_xy', 'gaussian', 'exponential', or 'external_{sph,cyl,rec}'
xs_point, ys_point, zs_point	0.0	Point-source location
source_zscale	NaN	Scale height of a 'gaussian' or 'exponential' source
radiation_angular_PDF_file	' '	Angular PDF for external illumination

The choice of par%source_geometry also drives the default par%geometry (sphere or cylinder) and the choice of the direct peel-off estimator. The external_* values describe an isotropic radiation field entering the system through a spherical, cylindrical, or rectangular boundary.
The scalar par%source_geometry describes a *single* internal source, or an external-only field. Two further configurations are available.

4.7 Multiple internal sources

Parameter	Default	Description
nsource	1	Number of internal sources (> 1 activates the multi-source path)
src_geometry(i)	'point'	Geometry of each source: 'point', 'uniform', 'uniform_xy', 'gaussian', 'exponential'
src_lum(i)	-999	Luminosity of each source (equal split of par%luminosity if unset)
src_x/y/z(i)	0.0	Position of each 'point' source
src_zscale(i)	-999	Scale height of a 'gaussian' or 'exponential' source

With nsource > 1 each packet picks its emitting component from the cumulative luminosity distribution, so the components are sampled in proportion to par%src_lum, and par%luminosity is set to their sum $\sum_i L_i$ (the image normalization therefore carries the correct total). Every packet still leaves its component isotropically with the grey dust properties par%albedo/par%hgg. The multi-source path carries no Stokes vector, so it is rejected at setup together with par%use_stokes.

4.8 Composition: internal sources with an external field

Parameter	Default	Description
ext_geometry	' '	Boundary the external field enters through when it *composes* with internal sources: 'sph', 'cyl', 'rec' (blank auto-selects from par%geometry/par%rmax)
ext_intensity	-999	Band mean intensity J of the isotropic external field; > 0 switches the composition on

An internal source (single or multiple) and an isotropic external field can illuminate the same system in one run. The composition is active when nsource ≥ 1 coexists with ext_intensity > 0 and par%source_geometry is *not* one of the external_* external-only shorthands (those remain the single-field path, unchanged). The luminosity entering the grid from an isotropic field of mean intensity J across an illuminated boundary of area $A_{\rm surface}$ is $\pi J A_{\rm surface}$, so the run

carries

$$L_{\rm tot} = L_{\rm int} + L_{\rm ext} = \sum_i L_i + \pi J A_{\rm surface}, \tag{1}$$

with $A_{\rm surface} = 4\pi(\texttt{rmax} \cdot d_{\rm cm})^2$ for `'sph'`, the six box faces for `'rec'`, and the barrel plus the two end caps for `'cyl'` ($d_{\rm cm}$ is the distance-unit conversion). Each packet is drawn internal-or-external with probability $L_{\rm int} : L_{\rm ext}$, the direct peel-off is routed to the internal or the external-boundary estimator according to the packet's origin, and the output is normalized to $L_{\rm tot}$. Because energy is additive, a composed run reproduces the internal-only run plus the external-only run. The composition is rejected at setup together with `par%use_stokes` and with the $(a, g)/\tau$ scans.

4.9 Dust and scattering

Parameter	Default	Description
`albedo`	0.3253	Dust single-scattering albedo a
`hgg`	0.6761	Henyey–Greenstein asymmetry g
`cext_dust`	1.6059e-21	Dust extinction per H nucleus [cm^2]
`scatt_mat_file`	' '	Tabulated Mueller matrix (`data/mueller_*.dat`); enables Stokes
`use_stokes`	.true.	Stokes polarization; forced off when no matrix file is given

Without `par%scatt_mat_file` the code falls back to the Henyey–Greenstein phase function with the scalar `par%albedo` and `par%hgg`.

4.10 Optical depth and density

Parameter	Default	Description
`taumax`	-999	Radial optical depth, center → outer edge (sets the density scale)
`tauhomo`	-999	Optical depth of the volume-equivalent homogeneous medium
`density_file`	' '	3-D density cube (FITS/HDF5); $\rho\kappa = n C_{\rm ext} d_{\rm cm}$
`reduce_factor`	1	Down-sampling factor for the density file
`centering`	0	Re-centering mode for the density file
`density_zscale`, `density_rscale`, `density_powerlaw_index`	NaN	Analytic density modulations for 'sphere', 'cylinder', and 'Plummer'

4.11 Peel-off observers

Parameter	Default	Description
`nobs`	1	Number of observers

Parameter	Default	Description
`nxim, nyim`	129	Image pixels
`dxim, dyim`	NaN	Pixel size (auto-sized to cover the grid if unset)
`distance`	NaN	Observer distance; renormalizes (obsx,obsy,obsz)
`inclination_angle, position_angle, phase_angle`	NaN	Observer angles [deg] (arrays)
`alpha, beta, gamma`	NaN	Euler rotation angles [deg] (alternative)
`obsx, obsy, obsz`	NaN	Explicit observer direction (alternative)

An image is peeled off toward each observer when `nxim` and `nyim` are positive, for up to `MAX_OBSERVERS = 181` observers in one run. Angle conventions are given in `docs/MoCafe_Geometry.pdf`.

4.12 Output control

Parameter	Default	Description
`file_format`	'hdf5'	'hdf5' or 'fits'; authoritative for the container
`out_file`	' '	Output base name (derived from the input name if empty)
`out_bitpix`	-32	FITS bitpix (−32 float32, −64 float64)
`save_direc0`	.false.	Also write the unattenuated direct image `Direct0`
`output_normalization`	'luminosity'	Output normalization
`sightline_tau`	.false.	Also write a sight-line optical-depth map for each pixel

5. Single-Run Scans

One Monte-Carlo run yields the scattered image over a whole grid of scattering parameters. Both scans are restricted to the no-Stokes Henyey–Greenstein path (`par%use_stokes = .false.`); combining either with Stokes is a fatal configuration. Neither scan consumes extra random numbers, so the forward random walk is untouched and the slice at the simulated point reproduces a stand-alone run bit for bit. The derivations are collected in `docs/MoCafe_agtau_scan.pdf`.

5.1 Albedo and asymmetry (a, g) scan — Seon (2010)

Parameter	Default	Description
`use_ag_list`	.false.	Enable the (a, g) scan → `scatt`(x, y, a, g)
`albedo_list`	NaN	Albedo grid (empty → 0.1 . . . 1.0)
`hgg_list`	NaN	Asymmetry grid (empty → 0.0 . . . 0.9)

Photons are tracked with the single simulated asymmetry g_0 = par%hgg (a value of 0.4–0.5 works well over the usual grid). Each packet carries the reweighting factors

$$A_a = a^k, \qquad W_g = \prod_{j=1}^{k} \frac{\Phi(\mu_j \,|\, g)}{\Phi(\mu_j \,|\, g_0)}, \tag{2}$$

after k scatterings, where Φ is the Henyey–Greenstein phase function, so the scattered peel-off becomes a 4-D image. The expensive quantities of each event (the optical depth toward the observer, the $1/r^2$ factor, the pixel binning) are computed once and reused over the whole $n_a \times n_g$ grid. The `Direct` and `Direct0` images are independent of a and g and stay 2-D.

5.2 Polychromatic optical-depth (τ) scan — Jonsson (2006)

Parameter	Default	Description
`use_tau_list`	.false.	Enable the τ scan → adds a τ axis
`tau_list`	NaN	Target taumax values (empty → $\sqrt{2}$-spaced $\tau/\tau_{\max}$ in $[0.5, 2]$; the reference taumax is inserted automatically)

Photons are tracked at the reference τ_0 — `par%taumax` if set, otherwise `par%tauhomo`, so the reference follows the grid normalization. For each target τ_t the medium is treated as grey-rescaled by $s_t = \tau_t/\tau_0$, and each free-path segment of optical depth τ_k multiplies the weight

$$W_\tau(t) \;\longleftarrow\; W_\tau(t)\, s_t\, e^{(1-s_t)\tau_k} \tag{3}$$

(Jonsson 2006, eq. 31), applied before the peel-off; the peel attenuation is $e^{-s_t \tau_{\rm obs}}$. The contribution therefore factorizes into an (a, g, τ) outer product evaluated once per event, and the two scans combine:

Flags	Scattered / direct images
`(neither)`	`scatt`(x, y)
`use_ag_list`	`scatt`(x, y, a, g)
`use_tau_list`	`scatt`(x, y, τ) and `direc`(x, y, τ)
`both`	`scatt`(x, y, a, g, τ) and `direc`(x, y, τ)

Unlike the (a, g) case the direct beam is τ-dependent and gains a τ axis, while `Direct0` stays 2-D. The variance grows as τ/τ_0 departs from unity, so keep `par%tau_list` within a factor of about 2–3 of the reference and place the reference near the middle of the list. The scan works with internal point and extended sources as well as with external illumination, whose direct peel-off estimators are τ-aware.

6. Clumpy Medium

With `par%grid_type = 'clump'` a sphere of radius `par%rmax` holds N discrete, non-overlapping spherical dust clumps; the space between clumps is vacuum, and `par%rmin` carves an inner cavity. Clumps are placed by Random Sequential Addition on a linked-list hash grid, and the transport uses ray–sphere intersection combined with an Amanatides–Woo

traversal of a CSR acceleration grid. Only the two ray-tracing routines are replaced: scattering and peel-off are shared with the Cartesian path. The medium is dust only.

Parameter	Default	Description
`rmax`	-999	Outer sphere radius (required)
`clump_radius`	-1	Radius of one clump (required)
`clump_f_cov / _f_vol / _N_clumps`	-1	Clump count — specify exactly one
`clump_tau0`	-1	Optical depth of one clump, center → surface ($\kappa = \tau_0/r_c$)
`clump_ndust / clump_nH`	-1	Dust or hydrogen density inside a clump ($\kappa = nC_{\rm ext}d_{\rm cm}$)
`rmin`	0.0	Inner cavity radius
`clump_fully_inside`	.true.	Require every clump to lie entirely inside the shell
`cone_opening`	0.0	Biconical placement half-angle [deg] (0 → the full sphere)
`clump_input_file`	' '	Load a clump population instead of generating one
`save_clump_info`	.false.	Write the generated clump table to `<base>_clumps`
`clump_radius_profile, clump_density_profile, clump_number_profile`	'constant'	'constant', 'powerlaw', 'gaussian', 'exponential', or 'file', each with `*_alpha` and `*_r0`
`clump_profile_file`	' '	Tabulated radial profile

If no clump opacity is given, it is back-solved from `par%taumax` or `par%tauhomo`. Random Sequential Addition is reliable up to a volume filling factor of about 0.35; beyond that the placement jams and MoCafe aborts with guidance. Populations can also be generated externally:

```
python python/make_clumps.py --single --rmax 1.0 --clump-radius 1.0 \
       --tau0 1.0 --out clump.fits        # one centered giant clump
python python/make_clumps.py --rmax 1.0 --clump-radius 0.1 --f-cov 1.0 \
       --tau0 1.0 --out clumps.fits       # uniform random population
```

Algorithm memo: `docs/MoCafe_clump.pdf`; examples in `examples/clump_sphere/`.

7. AMR Octree Mode

With `par%grid_type = 'amr'` an adaptive octree is read from a generic AMR file produced by the Python builders or converters. The tree, the neighbor table, and the leaf opacities are held in MPI shared memory, and the transport walks the octree through the neighbor table. As in the clumpy case only the ray tracers are replaced; grey dust scattering is leaf-independent and is shared with the Cartesian path. The medium is dust only, and `amr_type='ramses'` is rejected — convert the snapshot first.

Parameter	Default	Description
amr_type	'generic'	Only 'generic' is accepted
amr_file	''	Generic AMR file (.h5, .hdf5, .fits, .fits.gz, .dat)
dust_model	'global_dgr'	Leaf opacity model (below)
DGR	1e-2	Dust-to-gas ratio for 'global_dgr'
Z_global	0.0134	Uniform metallicity for 'laursen09' when there is no metallicity column
Z_ref	0.0134	Reference solar metallicity
f_ion_dust	0.01	Dust survival fraction in ionized gas (Laursen et al. 2009)

The dust scale is fixed by normalizing to par%taumax (a radial pole sight line) or par%tauhomo (the volume average); for an asymmetric model prefer par%tauhomo.

7.1 Generic AMR file format

A binary table (FITS or HDF5) or a text file, with the mandatory columns read by name (an index fallback handles legacy files): x, y, z (leaf-cell centers, in par%distance_unit), level (octree refinement level), and nH (hydrogen number density in cm^{-3}; the aliases dens and gasDen are accepted). Optional columns are metallicity, xHI (neutral fraction), and ndust (dust pseudo-density). Header keywords: BOXLEN, ORIGINX/Y/Z, and NAXIS2 (the number of leaves). Temperature and velocity columns are ignored.

7.2 Leaf dust models

dust_model	Required columns	Leaf opacity
global_dgr	nH	$n_{\mathrm{H}}\, C_{\mathrm{ext}}\, \mathrm{DGR}\, d_{\mathrm{cm}}$
from_file	ndust	$n_{\mathrm{dust}}\, C_{\mathrm{ext}}\, d_{\mathrm{cm}}$
laursen09	metallicity *and* xHI	$(Z/Z_{\mathrm{ref}})(n_{\mathrm{HI}} + f_{\mathrm{ion}} n_{\mathrm{HII}})\, C_{\mathrm{ext}}\, d_{\mathrm{cm}}$

The laursen09 model requires an explicit xHI column; deriving the neutral fraction from a temperature under collisional ionization equilibrium is deferred. Algorithm memo: docs/MoCafe_amr.pdf; examples in examples/amr_sphere/, examples/amr_tng/, and examples/amr_ramses/.

8. Converters and Python Tools

The octree builders and converters live under python/AMR_grid/ and all write the same dust-only schema; gas velocity and temperature are neither read nor written.

Tool	Description
`AMR_grid.py`	In-memory `AMRGrid` octree builder: uniform or density-gradient refinement, setters for the density, metallicity, neutral fraction, and dust density, and FITS/HDF5/text writers (with a module-level `write_leaves` for flat leaf arrays)
`make_amr_sphere.py`	Generic AMR dust sphere (uniform level or radially refined) for validation
`convert_ramses_to_generic.py`	Parses a native RAMSES output (`amr_*`/`hydro_*` Fortran-unformatted files) directly, without `yt`
`convert_illustris_to_generic.py`	Reads a local Illustris-TNG gas cutout or downloads one through the TNG.org API, converts comoving and h-scaled units to physical kpc and $n_{\rm H}$, builds an octree, and assigns `nH`, `metallicity`, and `xHI` by Voronoi nearest neighbor

```
python python/AMR_grid/convert_ramses_to_generic.py output_00042 \
       -o sim.fits --output-unit kpc --metallicity-index 6 --emit-ndust

python python/AMR_grid/convert_illustris_to_generic.py cutout.hdf5 \
       -o galaxy.h5 --grid-type amr --level-max 7 --boxsize 60 --emit-ndust
```

The RAMSES converter takes `--snapnum`, `--output-unit`, the hydro variable indices (`--density-index`, `--metallicity-index`, `--ionization-index`), and `--emit-ndust` (with `--Z-ref` and `--f-ion`) to precompute the `ndust` column. The Illustris converter takes `--grid-type` (`amr` or `uniform`), `--level-min` and `--level-max` (3 and 7), `--dens-threshold` (0.3), `--match-resolution`, `--center`, `--boxsize`, `--metallicity`, `--sfr-mask`, and the API arguments `--api-key`, `--simulation`, `--snap`, `--subhalo-id`.

Reading the output. `python/mocafe_io.py` is the Python counterpart of the Fortran I/O facade and is file-compatible with LaRT's `lart_io.py`. It exposes a `MoCafeFile` with the accessors `.scattered`, `.direct`, `.direct0`, `.total`, `.stokes`, and `.tau_dust`; the scan slicers `.scattered_at(a=,g=,tau=)` and `.direct_at(tau=)`; and a helper `find_mocafe_outputs(stem)` that resolves the suffixes of a run. Command-line verbs: `info`, `peek`, `convert`.

```
python python/mocafe_io.py info out_obs.h5
python python/mocafe_io.py convert out_obs.h5 out_obs.fits.gz
```

9. Output Products

`par%file_format` is authoritative: if the extension of `par%out_file` disagrees with it, MoCafe rewrites the extension (with a warning) so that the main file and every derived file share one container. Setting `out_file = '*.fits.gz'` alone is not enough — select the container with `par%file_format`. HDF5 mirrors the FITS HDU layout: each image HDU becomes a group named after its `EXTNAME` with a `data` dataset, and FITS keywords become group attributes

(`/Scattered/data` and `/Scattered/@XMAX`), written in the same section order.

Suffix	Written when	Contents
`_obs`	always	Observer images: `Scattered`, `Direct`, and (with save_direc0) `Direct0`, plus the scan axes when a scan is active
`_obs_tau`	sightline_tau	Sight-line optical-depth map
`_stokes`	use_stokes	Stokes I, Q, U, V images
`_NNN`	nobs > 1	One file per observer, numbered

With `par%file_format = 'hdf5'` these carry the `.h5` extension, and with `'fits'` the `.fits.gz` extension. The clump table written by `par%save_clump_info` follows the same rule.

10. Worked Examples

Directory	Shows
`examples/point_source`, `uniform_source`	Cartesian grids with a point and an extended internal source
`examples/external_sph`, `external_cyl`, `external_rec`	Isotropic external illumination through the three boundaries
`examples/external_Plummer`, `external_rexp`, `external_powerlaw`	Analytic dust-density profiles under external illumination
`examples/agtau_list`	The (a, g) and τ single-run scans, with a notebook
`examples/clump_sphere`	Clumpy medium (uniform, profile, cone, and file-loaded) validated against the smooth sphere
`examples/amr_sphere`	AMR octree sphere against the Cartesian reference
`examples/amr_tng`	Illustris-TNG cutout → converter → MoCafe AMR
`examples/amr_ramses`	RAMSES → converter → MoCafe AMR (template)

Each directory holds a `run.sh` showing the typical invocation and `plot_*.py` scripts that read the output with `python/mocafe_io.py`. There is no unit-test suite: validation is by reference comparison. The established checks are that a scan slice at the simulated point is bit-identical to a stand-alone run, that the clumpy and AMR media reproduce the homogeneous Cartesian sphere within Monte-Carlo noise, and that every Cartesian example stays bit-identical after any change to the shared peel-off routines.

10.1 A minimal input file

```
&parameters
 par%no_photons      = 1.0e6
```

```
 par%source_geometry = 'point'
 par%geometry        = 'sphere'
 par%rmax            = 1.0
 par%nr              = 65
 par%taumax          = 1.0
 par%albedo          = 0.4
 par%hgg             = 0.5
 par%use_stokes      = .false.
 par%distance_unit   = ''
 par%distance        = 1.0e5
 par%nxim            = 129
 par%nyim            = 129
 par%save_direc0     = .true.
/
```

11. Algorithm Notes

Photon packets and weights. Each packet carries a weight w, initially $L/N_{\rm phot}$ in units of `par%luminosity`. Scattering multiplies the weight by the albedo a instead of terminating the packet, so packets are immortal in the scattering sense and every history contributes to all scattering orders.

Forced first scattering and the reduced weight. With `par%use_reduced_wgt` (the default) the first interaction is forced to occur inside the medium. The optical depth τ_0 from the birth point to the grid boundary is obtained by a ray trace, the first free path is drawn from the truncated exponential

$$\tau = -\ln\left[1 - U\,(1 - e^{-\tau_0})\right], \qquad U \sim \mathcal{U}(0,1), \tag{4}$$

and the weight is reduced by the escape-corrected factor $1 - e^{-\tau_0}$. The escaping fraction is accounted for separately by the direct peel-off, so the estimator stays unbiased while the variance of the scattered image at low optical depth drops sharply.

Peeling-off (next-event estimation). At every scattering site a deterministic contribution is sent toward each observer: the packet weight times the phase-function value for the scattering angle toward the observer, times $e^{-\tau_{\rm obs}}$ with $\tau_{\rm obs}$ integrated along the line of sight to the system boundary, times the $1/r^2$ and solid-angle factors of the observer geometry. The unscattered beam is treated the same way at birth and is tallied as `Direct` (attenuated) and, optionally, `Direct0` (unattenuated). The observer-side optical depth is integrated through the same ray-tracing procedure pointer as the transport, which is why the Cartesian, clumpy, and octree media share one peel-off implementation.

Scattering. Without a Mueller-matrix file the deflection cosine is drawn from the Henyey–Greenstein distribution with the asymmetry `par%hgg` and the azimuth is uniform. With `par%scatt_mat_file` the tabulated matrix elements

S_{11} through S_{34} are normalized by the S_{11} integral and an alias table gives $O(1)$ sampling of $\cos\theta$; the Stokes vector is rotated into the scattering plane, multiplied by the Mueller matrix, and rotated back, and the peel-off uses the same matrix evaluated at the observer direction.

Parallelism and accumulation. In master–slave mode rank 0 hands out batches of `par%num_send_at_once` photons, which suits configurations where the cost of a photon varies strongly (deep clouds, external illumination); equal-share mode instead gives rank r the photons $r+1, r+1+n_{\rm proc}, \ldots$ with lower overhead and exact reproducibility. The observer images are combined with an MPI reduction and normalized on rank 0 before being written. Read-only arrays — the scattering matrix, the density grid, the clump table, and the octree — are held in MPI-3 shared memory, one copy per node.

Full derivations and conventions: `docs/MoCafe_Geometry.pdf` (rotation angles and image-plane geometry), `docs/output_definitions.pdf` (output image definitions), `docs/MoCafe_agtau_scan.pdf` (the two scans), `docs/MoCafe_clump.pdf` (the clumpy medium), and `docs/MoCafe_amr.pdf` (the octree).

References

- Seon, K.-I. & Draine, B. T. 2016, ApJ, 833, 201 — dust radiative transfer and scattered-light modeling by the author.
- Seon, K.-I. & Kim, C.-G. 2020, ApJS, 250, 9 — Monte-Carlo radiative transfer in a turbulent medium.
- Seon, K.-I. 2010, PKAS, 25, 177 — the (a, g) single-run scan.
- Jonsson, P. 2006, MNRAS, 372, 2 — the polychromatic optical-depth scan (SUNRISE).
- Laursen, P., Sommer-Larsen, J., & Andersen, A. C. 2009, ApJ, 704, 1640 — the metallicity- and ionization-dependent dust model.
- Amanatides, J. & Woo, A. 1987, Eurographics '87 — fast voxel traversal.
- Henyey, L. G. & Greenstein, J. L. 1941, ApJ, 93, 70 — the scattering phase function.